Task D1 — Dataset Generation and Model Training

Dataset, MLP training and validation | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component D, Module 4

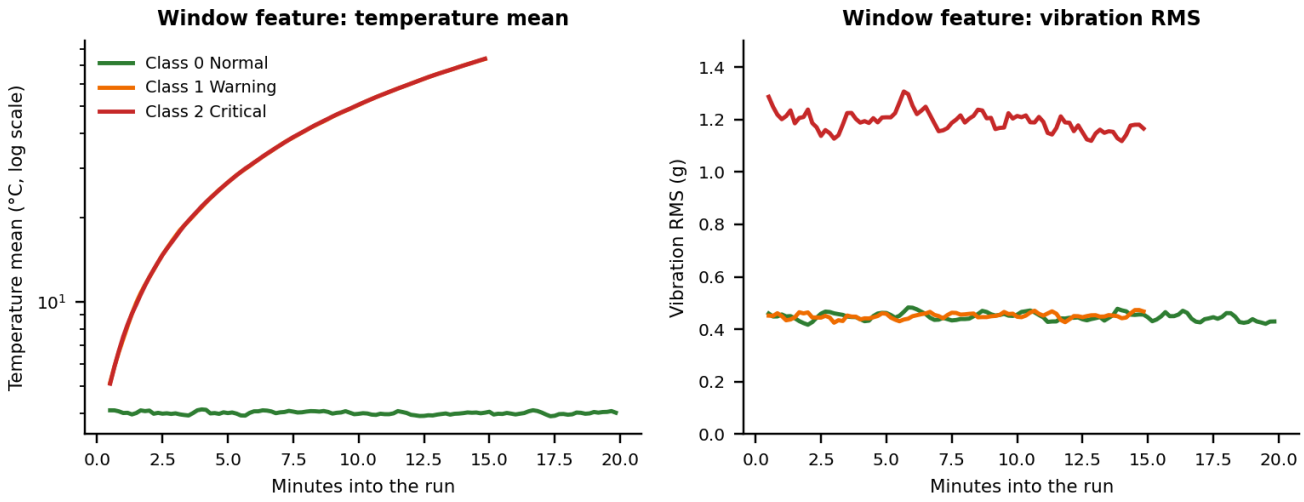
At a glance

Item	Result
Dataset	291 labelled windows: Normal 117, Warning 87, Critical 87 (targets: about 120, 90, 90)
Model	MLP 6 → 32 → 16 → 3, ReLU, softmax output, 803 parameters
Required validation (20% split)	**98.25%** (56/57 windows; 95% interval 90.7% to 99.7%). **Passes the 88% threshold.**
Wider checks	99.2% mean over 50 random splits (lowest 94.7%, none under 88%); 99.3% five-fold; 99.7% on independent runs
Two findings to act on	The model misses some faults that start mid-run, and the table's labels call a 70 °C drift “Warning” (Section 5)

1 Dataset generation

Class	Label	Simulator mode	Duration	Target	Generated	Seed
0	Normal	--anomaly none	20 min	~120	117	101
1	Warning	--anomaly temp_drift	15 min	~90	87	102
2	Critical	--anomaly combined	15 min	~90	87	103

- The simulator loop from Task C1 was run once per mode for the stated duration. Its messages went through the Task C2 pipeline, giving one 6-value feature vector per 30 s window every 10 s, normalised with `training_stats.npy`.
- A 20-minute run gives 117 windows rather than 120 because a window needs its full 30 s (1,200 s → 117; 900 s → 87).
- The run is in-process on an accelerated clock, so it needs no broker. `generate_dataset.py` documents the equivalent live commands for recording from Mosquitto.



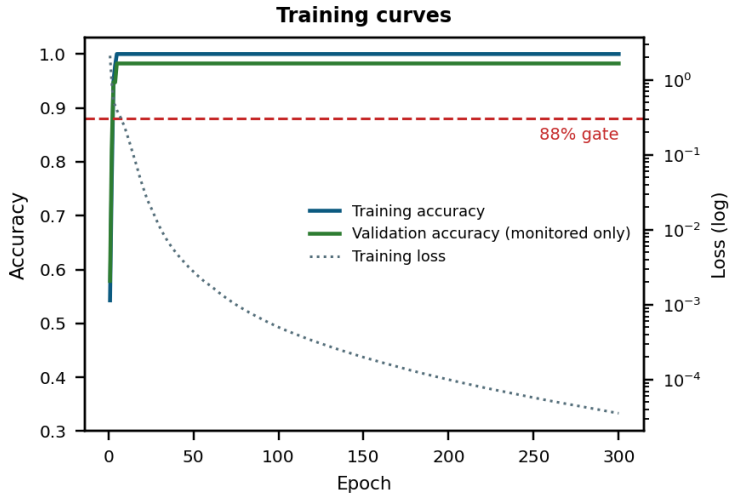
In the left panel the Warning line sits underneath the Critical line, because both runs use the same drift. Vibration RMS is what separates Warning from Critical.

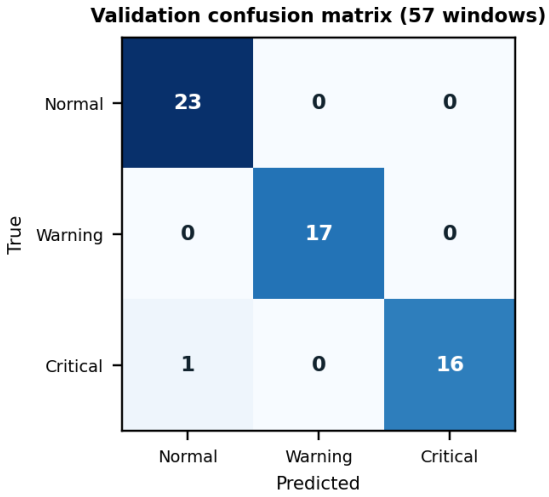
2 Model and training

Setting	Value
Architecture	6 inputs → 32 ReLU → 16 ReLU → 3 softmax outputs (803 parameters)
Input	The 6 normalised features from Task C2, using the saved training_stats.npy
Split	Stratified 80 / 20: 234 training and 57 validation windows (23 Normal, 17 Warning, 17 Critical), seed 42
Training	Adam, learning rate 0.001, batch 32, 300 epochs, cross-entropy loss, He initialisation (first layer scaled by 0.05)
Validation use	Monitored only; it never picks the epoch or any setting
Gate	`train_model.py` exits with an error and saves no model if validation accuracy is 88% or below
Implementation	Pure numpy, no framework needed; weights saved as plain arrays in model_weights.npz; scikit-learn MLP (32, 16) used as a cross-check

3 Results on the required validation set

Class	Windows	Precision	Recall
Normal	23	95.8%	100.0%
Warning	17	100.0%	100.0%
Critical	17	100.0%	94.1%
Overall accuracy	57		**98.25%**



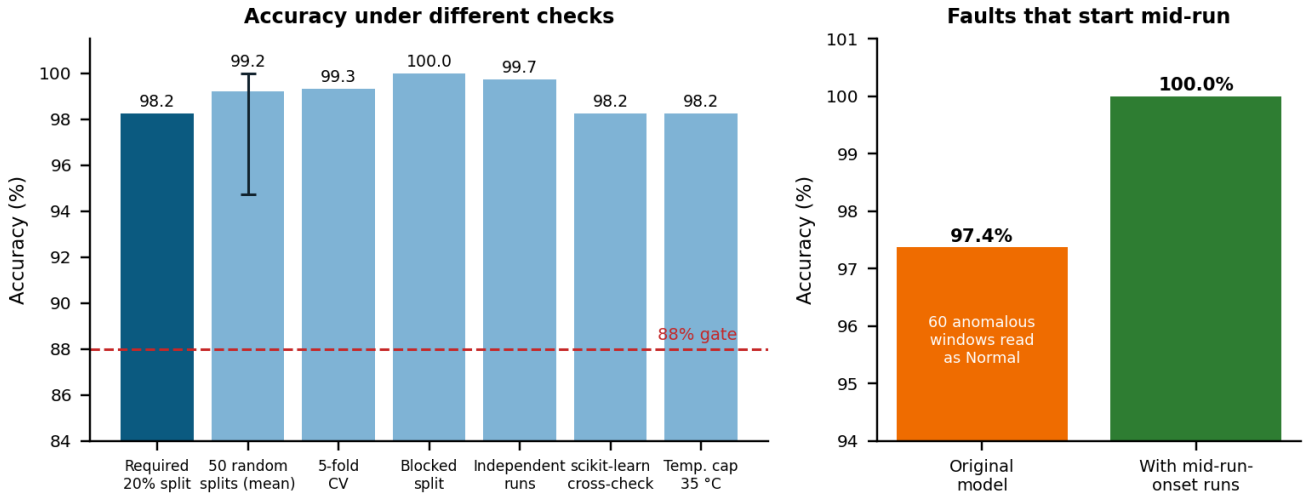


- The one error is a Critical window read as Normal: the very first window of the combined run, where the drift has barely started (mean temperature about 5.1 °C). The model gave Normal with near certainty even though vibration was 80σ above normal.
- With 57 validation windows, each error is 1.75 points, so the interval is wide (90.7% to 99.7%). The next section checks how stable the result is.

4 How far the number can be trusted

Overlapping windows (each shares 20 of 30 s with its neighbour) make a random split optimistic, so the result was checked six more ways.

Check	Accuracy	Note
50 random stratified splits	99.23% ± 1.32	Range 94.7% to 100%; none at or below 88%
Five-fold cross-validation	99.32%	Folds from 98.2% to 100%
Blocked split (last 20% of each run)	100.0%	57/57; a 2-window gap prevents overlap with training
Independent runs (5 new seeds)	99.73%	1,455 windows; lowest set 99.3%; 4 Critical windows read as Normal
scikit-learn MLP (32, 16), same split	98.25%	Same accuracy as the numpy model (56/57)
Temperature capped at 35 °C	98.25%	Physically plausible ceiling instead of unbounded drift



The left axis starts at 84% so differences are visible. The right panel is explained in Section 5.

5 Fitness for cold-chain use: two findings

The model clears the 88% threshold with a wide margin. For a pharmaceutical deployment, two things the single number hides are worth fixing before the model is converted and deployed.

Finding 1: faults that start mid-run can be missed

Test	Original model	Trained with mid-run-onset runs added
Standard runs (fault from t = 0), 5 independent sets	99.73%	100.00%
Runs where the fault starts at 80–320 s, 5 independent sets	97.37%	100.00%
Anomalous windows read as Normal (of 1,520)	60 (3.9%)	0
Required validation split	98.25%	100.0% (57/57)

The training runs start every fault at t = 0, so they contain no windows between “normal” and “fully drifting”, and the model behaves unpredictably there. Adding runs with `--anomaly-start` fixes it. This model is saved as `model_weights_onset.npz` (`train_model.py --augment-onset`) and also passes the gate. **I recommend carrying it forward.**

Finding 2: the table's labels disagree with the Task C class definitions

The assignment labels by simulator mode, and the drift is unbounded, reaching about 76 °C. By the Task C thresholds (Warning is 1–3 °C off setpoint, Critical is more than 3 °C), 86 of the 87 “Warning” windows are already Critical. The model learns what the table says, so a pure temperature breach reads as Warning.

Truth by Task C thresholds (independent runs)	Windows	Predicted Normal	Predicted Warning	Predicted Critical
Normal	351	351	0	0
Warning	3	0	3	0
Critical	519	3	258	258

Agreement with threshold-based labels is 70.1%: 258 of 519 temperature-breach windows are called Warning where Task C says “immediate escalation required”. Options: (1) add a rule after the model that forces Critical when mean temperature exceeds setpoint + 3 °C, (2) relabel by thresholds, or (3) generate shorter drift episodes so Warning contains 1–3 °C windows. I recommend (1) with (3). Please choose.

6 Files and how to run

File	Purpose
generate_dataset.py	Runs the simulator in the three modes and writes training_dataset.npz and .csv
train_model.py	Trains the 6-32-16-3 MLP, applies the 88% gate, saves model_weights.npz and model_metadata.json
model_weights.npz	The model required by the task (original data)
model_weights_onset.npz	Recommended model (mid-run-onset runs added); metadata in model_metadata_onset.json
validation_checks.py	The wider checks in Section 4 and the onset and label tests in Section 5
test_training.py	21 tests; the full project suite is 76 tests, all passing

```
python3 preprocessing.py make-stats          # training_stats.npy (Task C2), once
python3 generate_dataset.py                  # 291 labelled windows
python3 train_model.py                       # exits with an error below 88%
python3 train_model.py --augment-onset --out model_weights_onset.npz --meta model_metadata_onset.json
python3 validation_checks.py                 # wider checks (about 1 minute)
python3 -m unittest -v test_training
```

7 Assumptions and items to confirm

- **Labelling policy.** Which option do you want for Finding 2? The required model follows the table as written.
- **Which model to carry forward** to conversion and deployment: the required one or the onset-trained one (recommended).
- **Simulated data.** Near-perfect accuracy mostly reflects how separable the simulator's modes are. It is not evidence of performance on real trucks, so real data should be added when available.
- **Framework.** The model is plain numpy. For conversion I can load the saved weights into an equivalent Keras or PyTorch model and check the outputs match; tell me which toolchain you will use.
- **Seeds and temperature.** Seeds 101–103 are fixed for reproducibility. Temperature drift is unbounded as in Task C1; use `--temp-cap 35` for a plausible ceiling.